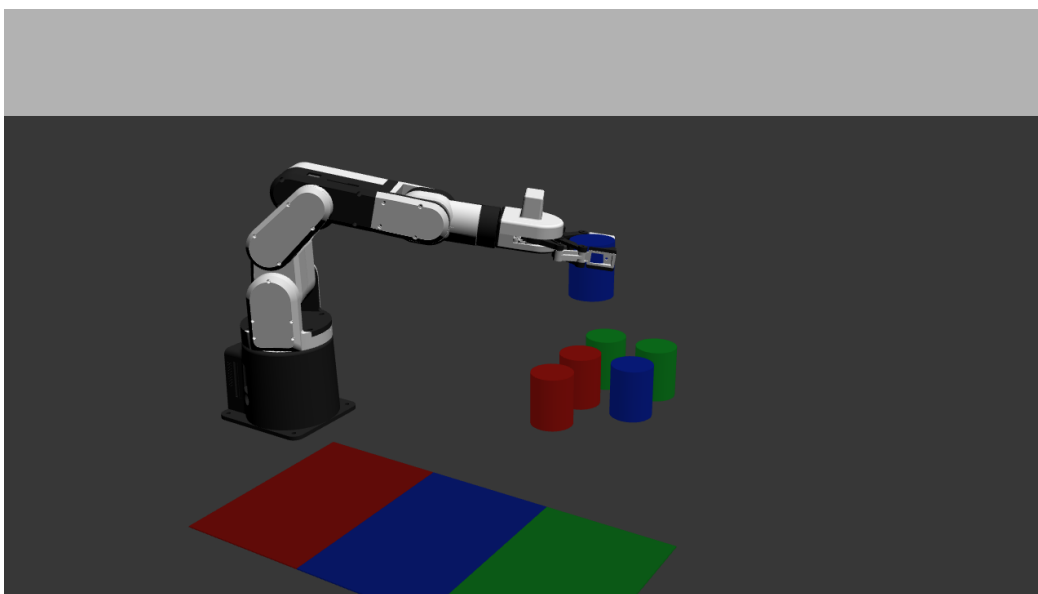


EXPERIMENT 3

桌面物体自动分类整理

个人实验报告 · mechArm 270 仿真调试记录

Personal laboratory report



姓名	杨牧青
学号	24020036076
小组与课程	第 17 组 · 机器人学集成小组项目 I

ROS 2 HUMBLE · GAZEBO · mechARM 270

SEPTEMBER 2026

1 实验基本要求

1.1 任务目标与考核重点

实验三的任务是让机械臂依据视觉结果完成桌面物体自动分类。程序从相机图像和检测数组中得到物体类别、置信度和位置，将检测框中心映射到固定取物网格，再依据类别选择对应的分类垫，依次完成接近、抓取、抬升、搬运、释放和结果记录。处理完一个目标后，任务节点继续处理后续目标。

任务验收包括自动决策、异常分支和证据完整性：运行时由系统自动确定网格和料盒；低置信度、网格外、重复占用和不可达目标生成可解释的跳过或停止结果；识别、抓取、放置和最终状态通过日志、画面和位姿记录复查。两类六物体的仿真基线要求至少正确整理五个目标。

1.2 实验环境与参数

本次个人工作在 Ubuntu 22.04、ROS 2 Humble 和 Gazebo Classic 11 环境中完成，使用 Python/rclpy 组织场景监视、任务规划和执行节点。机械臂为 mechArm 270 六自由度模型，夹爪通过 ros2_control 控制；俯视相机输出 640×480 图像，检测结果以 Detection2DArray 形式进入任务节点。

项目	最终采用的设置
场景	0.72 m×0.52 m 桌面；3 行 2 列六格取物区；红蓝绿三类圆柱
分类垫	红(0.14,-0.12)、蓝(0.26,-0.12)、绿(0.38,-0.12) m，横向排列
识别判断	置信度阈值 0.80；最近网格距离阈值 48 px
示教记录	manual_animation.json；27 个航点；34 步自动播放；每步 2.0 s
验收记录	任务 CSV、执行 CSV、异常日志、GUI 过程图和最终位姿

1.3 本次完成情况

我参与完成了仿真场景调整、手动示教、自动序列组织、抓取附着检查和结果核对。红蓝两类六个目标的对象级记录显示识别、抓取和放置均为 6/6，最终状态为 passed；红蓝绿三分类扩展完成了分类垫对齐、绿色目标航点和 34 步播放组织，现有归档材料包含场景和序列证据，三分类对象级成功率待独立 CSV 归档后再统计。

2 个人分工

我的工作集中在将检测和任务规划结果转化为稳定、可复查的机械臂动作，具体按“启动环境—手动示教—参数调试—自动播放—证据核对”的顺序展开。

- 检查 Gazebo 世界、控制器、相机和任务节点的启动关系，统一控制接口和进程实例。
- 使用手动控制台调节关节、夹爪和动作时间，记录每个网格的 above、pick、lift、release 航点。
- 核对物体几何尺寸、分类垫位置、置信度阈值和像素网格参数，使画面、规划和执行使用同一套坐标关系。
- 组织六个目标的自动播放子序列，检查 pick 和 release 航点的动作语义，并复核 manual_animation.json。
- 根据 attached、release、settled 和最终位姿等证据综合判定动作成功状态，建立可复查的验收依据。

2.1 场景与启动组织

我先检查场景文件中的桌面尺寸、六个圆柱和三块分类垫是否全部在相机视野内，再确认世界坐标与任务节点使用的像素网格一致。手动调试时只启动世界、控制器和手动面板；自动验收时才启动任务播放器，并确保 Gazebo master、控制器和播放器各只有一个实例。

2.2 手动示教与航点记录

我使用关节滑块和夹爪参数逐点寻找安全高度、接近位置、抓取位置、抬升位置和释放位置。调试采用单变量方法，每次修改一个目标或一个动作阶段，确认末端与圆柱的相对位置后保存航点；航点记录同时标注接近、闭合、抬升和释放阶段，便于后续定位。

2.3 参数与抓取验证

我重点检查圆柱是否位于两指之间、夹爪闭合后目标是否随末端抬升，以及释放前后 X/Y 是否保持一致。对于仿真中的逻辑附着，我把“目标高度超过 0.075 m”作为画面取证条件，并同时检查 attached 事件，用两项证据区分夹爪闭合和成功抓取。

2.4 自动序列与日志整理

我先运行单个目标，再运行一个目标的完整子序列，最后才组合六个目标。航点名中包含 pick 的位置触发抓取，包含 release 的位置触发释放，其余位置只表示运动。自动运行后，我对照对象编号、网格编号、类别、事件和最终位姿，整理出可与截图相互印证的 CSV 和异常记录。

2.5 集成验收

最后我用红蓝六目标场景进行对象级基线检查，再单独检查红蓝绿扩展场景的分类垫排列和自动播放顺序。扩展结果按场景证据、接口工作和对象级数据分别归档，红蓝基线的 6/6 结果单独列示。

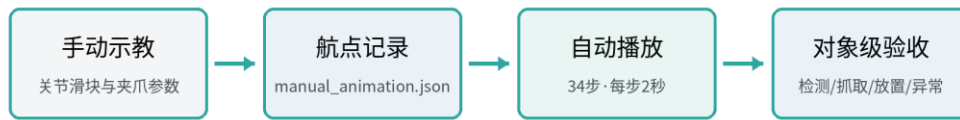


图 1 我的工作手动示教到对象级验收链路中的位置

3 项目实施原理

3.1 系统组成与数据流

从个人调试角度看，系统可以分为场景、感知、规划、执行和证据五层。Gazebo 提供桌面、圆柱、分类垫和机械臂模型；监视节点发布 Image 和 Detection2DArray；任务节点完成置信度筛选、网格匹配、类别查询和队列去重；执行节点读取队列并完成抓取和释放；GUI、事件日志、CSV 和最终位姿共同组成验收证据。

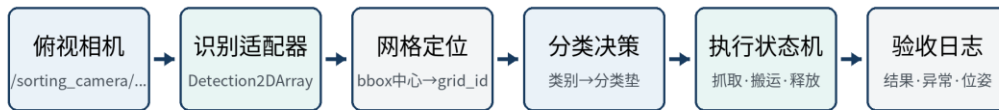


图 2 仿真系统中识别、网格定位、分类和执行模块的连接

3.2 网格定位与分类决策

我参与核对的核心关系是“检测框中心—最近网格—类别—分类垫”。任务节点把检测框中心投影到图像坐标，并与 3 行 2 列的固定格点比较；最近格点距离不超过 48 px 才生成 grid_id。类别 red、blue、green 通过显式配置表分别指向 red_sort_bin、blue_sort_bin 和 green_sort_bin。未知类别、低置信度结果和同一网格重复检测被标记为 skipped，并生成对应原因。

红蓝两类定量基线的六个占用网格

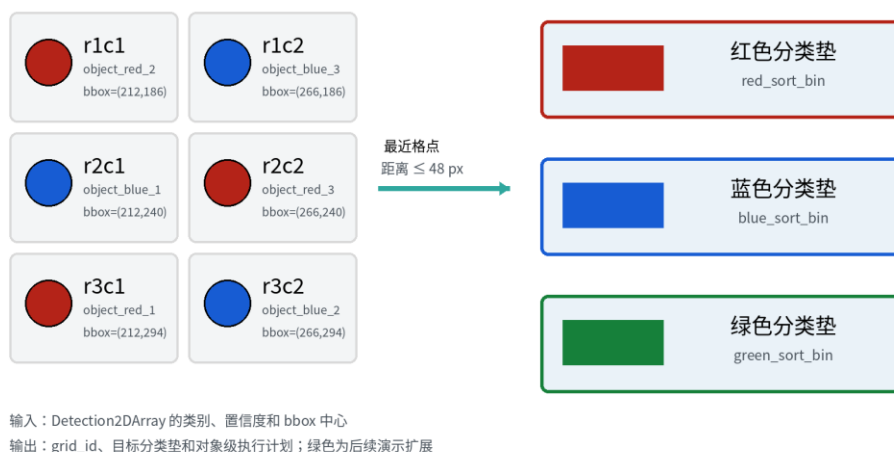


图 3 我参与核对的六个网格、类别和分类垫关系

3.3 抓取与搬运动作链

一个可复现的抓取被拆成六个阶段：先到安全高度 above，再下降到 pick，闭合夹爪后检查末端与目标距离，确认附着后 lift，最后搬运到分类垫上方。搬运过程中保存并更新目标相对夹爪的偏移，使目标持续跟随末端运动。

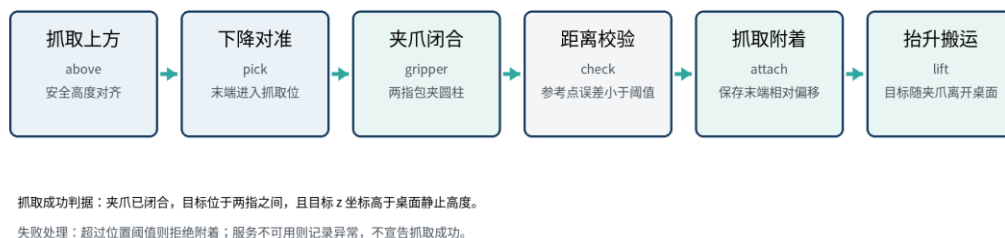
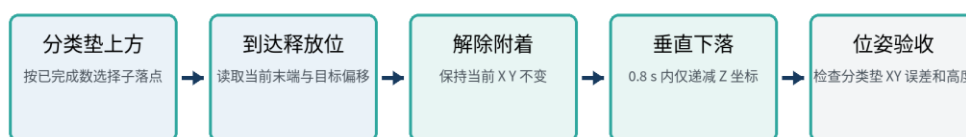


图 4 我调试的接近、闭合、检查、附着和抬升动作

3.4 垂直释放与落点检查

释放阶段读取当前末端位置，保持 X/Y 对齐并沿 Z 方向下降；解除附着后等待目标稳定，再读取最终位姿。对于同一颜色的多个目标，为它们分配不同落点，形成清晰的放置顺序。



为什么不写死坐标：释放起点由当前实际夹爪位姿决定，避免目标瞬间横移到分类垫中心。

同色不重合：同一分类垫为第 1 个和第 2 个目标预留不同子位置。

图 5 垂直解除附着和落点检查流程

3.5 航点语义与状态证据

manual_animation.json 保存已经验证过的航点和动作顺序，任务节点负责类别决策。播放器根据包含 pick、release 的航点名触发相应语义，其余航点承担运动。一次任务通过的条件包括 GUI 几何关系、状态反馈、attached/release 事件和最终位姿检查，四类证据共同确认结果。



图 6 手动控制台中的关节、夹爪和航点列表



图 7 六个目标的自动播放序列

4 项目中遇到的问题和解决方法

本节沿用实验二个人报告的写法，将每个问题记录为“现象—调查—处理—验证”，判断依据具体落实到坐标、日志事件和场景状态。

4.1 默认姿态导致夹爪偏离目标

现象：三分类场景沿用默认关节姿态后，夹爪没有落在圆柱中心，下降时会擦到目标边缘。调查：我在 GUI 中暂停于 above 和 pick，比较末端与圆柱中心的相对位置，并逐个检查六个网格的接近方向。处理：以已验证的 A-B 动作为参照，为每个网格重新调节 above 和 pick 航点，先单目标验证，再并入总序列。验证：修正后前几个目标能够按顺序对准并抬升，航点名称和控制台记录保持一致。

4.2 夹爪闭合但物体没有离桌

现象：日志显示 pick 已触发，但圆柱仍停在桌面，夹爪抬升后目标没有跟随。调查：我将动作拆成闭合、距离检查和抬升三段，发现播放器只发送夹爪命令，没有保存目标与末端的相对位姿。处理：增加目标选择和附着跟随逻辑，在搬运中持续更新相对偏移，并把目标高度超过 0.075 m 作为画面取证条件。验证：夹爪闭合后目标位于两指之间并离开桌面，同时出现 attached 事件。

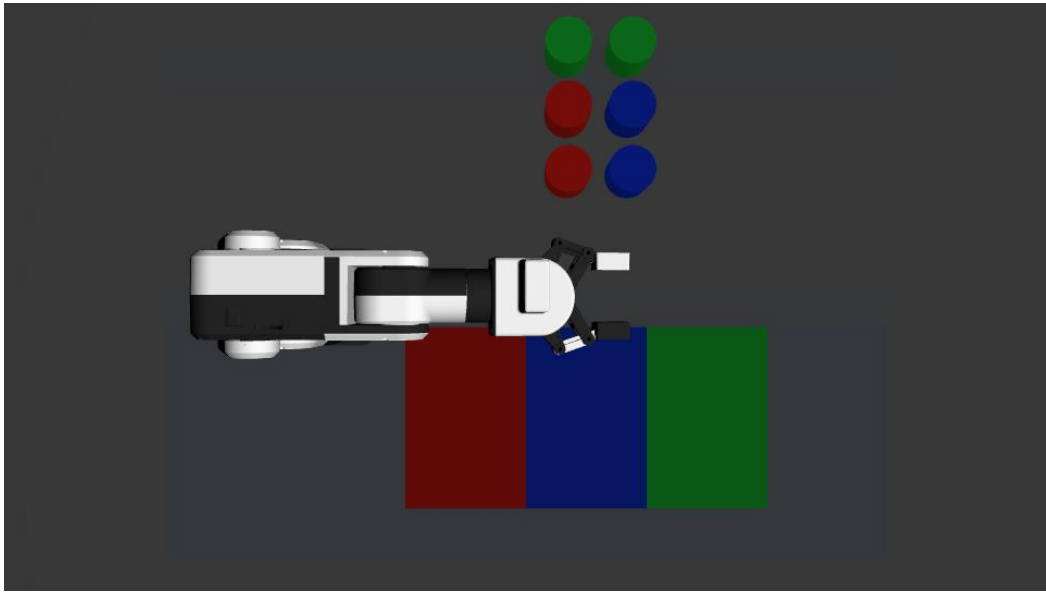


图 8 夹爪闭合、目标位于两指之间且已离开桌面的验证画面

4.3 释放跳变和同色目标重叠

现象：目标直接写到分类垫中心时，画面缺少真实释放过程；同色目标会落在完全相同的位置。调查：我分别观察释放前后的末端坐标、目标高度和横向位移，确认横向跳变会让落点受瞬时速度影响。处理：从当前末端位置开始，保持 X/Y 不变，解除附着后只沿 Z 方向下落，并给同色目标分配不同落点。验证：画面连续呈现对准、解除附着、下落和稳定四个阶段，最终落点形成清晰间隔。

4.4 世界坐标和像素网格不同步

现象：调整场景坐标后，任务节点只识别到少数目标，完整计划中出现空缺。调查：我对比世界中三行目标的位置和监视节点使用的像素行坐标，发现场景已经变化而旧网格参数仍被加载。处理：根据投影关系重新计算三行像素坐标 186、240、294，并清理旧实例后重新启动。验证：六个目标都得到唯一 grid_id，任务队列的对象顺序与俯视图一致。

4.5 重复启动导致动作不发送

现象：界面看起来正常，但播放器发送动作后机械臂没有响应，或者同一动作被重复触发。调查：我检查 world、controller 和播放器进程，发现旧 Gazebo master 与新实例同时存在。处理：结束旧进程，只保留一个世界；手动 GUI 已打开时直接连接播放器。验证：控制器只收到一套命令，单目标测试和整序列播放都按预期推进。

4.6 节点启动日志报错

现象：任务节点启动后立即退出，终端提示日志调用参数不兼容。调查：我定位到 Humble Python logger 的调用方式与当前代码不一致，错误发生在真正进入任务状态机之前。处理：改用兼容的 f-string 日志写法，重新构建并在干净终端中启动节点。验证：节点保持运行并继续发布状态，抓取问题可以在稳定的启动环境中单独分析。

问题	我的判断依据	修复后的检查
姿态不准	末端与目标的相对位置	抓取距离与对准画面
未附着	目标高度和 attached 事件	目标随夹爪抬升
释放异常	释放前后 X/Y、Z 变化	垂直下落和分离落点
网格错误	投影像素与 grid_id	六个目标唯一编号
运行冲突	world/controller 数量	单实例播放
节点退出	终端错误位置	节点持续发布状态

5 仿真结果与个人总结

5.1 红蓝基线对象级结果

红蓝两类六个目标的对象级 CSV 是本次量化验收的主要依据。六个目标的识别置信度均为 0.99，抓取和放置结果均完成，最终状态为 passed，success_count=6。这个结果满足任务书“至少正确整理 5 个物体”的红蓝基线要求。

类别	对象数	识别	抓取	放置	结论
蓝色圆柱	3	0.99	3/3	3/3	通过
红色圆柱	3	0.99	3/3	3/3	通过
合计	6	6/6	6/6	6/6	通过

5.2 三分类扩展结果的边界

我参与完成了红蓝绿三分类场景、分类垫对齐和自动播放序列。扩展阶段证明新增绿色类别可以复用检测消息、网格定位、类别映射和执行接口；当前归档材料包含场景、航点和播放证据，三分类对象级 CSV 完成归档后再统计对应成功率。



图 9 三分类扩展中识别、定位、分类和执行模块的连接

5.3 证据链与调试工具

GUI 用来检查末端、夹爪和目标的几何关系；状态反馈用来判断当前处于扫描、规划、执行还是释放阶段；事件日志用来确认 attached、release 和 settled 是否按顺序发生；最终位姿用来判断落点；对象级 CSV 则把这些观察按 object_id 汇总。多层证据互相印证后，才能区分“动作完成”和“任务成功”。



验收原则：单独的动画或单条“完成”日志都不足以证明整理正确，必须将过程、事件和最终落点互相印证。

图 10 我整理的过程画面、状态、事件和 CSV 证据链

5.4 个人总结

这次实验让我认识到，机械臂仿真要求同时保证关节运动、场景模型、控制接口、任务逻辑和验收证据的一致性。一个动作只有在“目标被正确选中、夹爪与目标相对位置满足条件、目标高度发生变化、事件顺序完整、最终位姿落在分类区域”时，才具备可复查的成功依据。这个认识改变了我对仿真结果的判断方式：我开始把画面、状态和数据看成同一件事的不同观测，而不是只看机械臂是否完成了一段轨迹。

在调试过程中，我逐渐形成了固定的排查顺序。第一步是把问题缩小到扫描、网格定位、接近、抓取、搬运、释放或验收中的一个阶段；第二步是用单目标和手动面板复现现象，记录末端位置、目标位置、夹爪状态和当前航点；第三步是一次只修改一个变量，例如像素网格、抓取高度、夹爪开度、释放高度或动作时间；第四步是用事件日志、最终坐标和完整序列复核修改结果。网格错位、释放跳变、目标未附着和重复启动等问题，都按照这套顺序得到定位。

- 用阶段化思路分析动作：先确认任务进入了哪个状态，再检查该状态的输入、输出和退出条件。

- 用几何关系验证抓取：同时观察指垫中心、圆柱中心、目标高度和相对位姿，确定运动轨迹与对象状态一致。
- 用最小实验减少干扰：先单点、再单目标子序列、最后整序列，降低多个问题同时出现时的判别难度。
- 用对象级记录闭环：把 object_id、grid_id、类别、事件、最终位姿和异常码对应起来，使每个目标都能单独复核。
- 用干净环境复现：控制 Gazebo world、控制器和播放器实例数量，保证每轮测试的启动条件相同。

这次实验还让我熟悉了模块之间的接口责任。任务节点负责生成对象顺序和分类目标，执行节点负责完成动作与反馈，场景和日志负责提供复核依据。我的工作位于任务规划结果与仿真验收之间，需要把抽象的任务项落实成可执行的姿态、航点和状态检查。这使我更加重视命名、参数记录、日志字段和图片说明的完整性，因为它们直接影响小组成员之间的协作和后续复现。

仿真中的逻辑附着提升了动作重复性，也让我看清了仿真与真机之间的差异。真实系统还需要重新标定相机映射、夹爪摩擦、接近高度、闭合速度、释放等待和急停距离。后续迁移到真机时，我会沿用“低速单目标—检查识别—检查抓取—检查释放—加入第二类—扩展到六目标”的顺序，每一步保存对象级记录，再决定是否进入下一阶段。

6 结论

我完成了实验三仿真执行侧的主要个人工作，并把个人贡献落实到场景、航点、状态和证据四个层面。通过场景调整、手动示教、动作修复、自动播放和对象级复核，我参与建立了从任务规划结果到机械臂动作，再到实验结论的完整链路。红蓝两类六个目标的对象级结果达到 6/6，三分类扩展完成了场景、分类垫和自动序列的集成。

6.1 个人工作完成情况

工作模块	完成内容	对应结果
场景与启动	调整六格桌面、红蓝绿圆柱和三块分类垫；检查 world、控制器、相机和播放器的启动关系	三分类场景稳定加载，单实例运行
航点与动作	建立 above、pick、lift、release 等语义航点，组织六目标子序列和统一动作节拍	manual_animation.json 保存 27 个航点，自动播放 34 步
抓取与释放	校准末端参考点，加入距离检查、目标跟随、垂直释放和同色不同落点	目标能够离桌、跟随搬运并落入对应分类垫
问题修复	处理姿态偏差、网格不同步、重复启动、节点日志报错和释放跳变	单目标与整序列复跑通过
结果核对	对照 GUI、状态、事件、最终位姿和对象级 CSV 复核结果	红蓝基线对象级结果为 6/6

6.2 个人能力收获

我掌握了 Gazebo 场景参数与 ROS 2 控制器之间的联调方法，能够从世界坐标、像素网格、末端姿态和对象状态四个角度检查同一动作。通过手动控制台和 JSON 航点文件，我熟悉了从单点测试到完整序列的动作组织方式；通过状态主题、事件日志和对象级 CSV，我能够把一次运行拆分为可定位、可比较的阶段。

在协作方面，我进一步理解了统一接口和统一命名的重要性。任务节点输出的 `object_id`、`grid_id` 和 `target_bin`，必须与执行节点的航点、日志和最终位姿相互对应，报告中的图片和表格也要能够回到具体对象。这样的记录方式提高了问题定位效率，也使个人调试结果能够直接支持小组验收。

6.3 后续改进计划

后续真机工作将先完成相机单应性标定和桌面网格标记，再用单个目标验证识别、PICK、抬升、PLACE 和回 HOME。确认抓取高度、夹爪开度、释放等待和急停策略后，再加入第二类目标并扩展到六个目标。每个阶段继续保存真实类别、预测类别、置信度、网格编号、抓取结果、放置结果和异常码，便于与当前仿真字段对照。

从个人方法上，我会继续保持“阶段定位、单变量修改、最小案例复现、完整数据复核”的工作习惯，并补充运行 ID、日志轮转、连续帧稳定判定和异常恢复记录，使后续实验具备更好的可重复性和安全性。

6.4 GitHub 同步记录

我参与整理的实验三工程、证据和报告已同步到 GitHub 仓库：<https://github.com/9161942-boop/robotics-experiment-3>。

仓库中保留 ROS 2 工程、Gazebo 场景、manual_animation.json、对象级执行记录、过程截图以及小组和个人报告。提交历史截图用于说明材料已经进入公开仓库；个人报告中的场景调整、航点示教、问题修复和证据归档均能在报告文件与仓库目录之间对应。

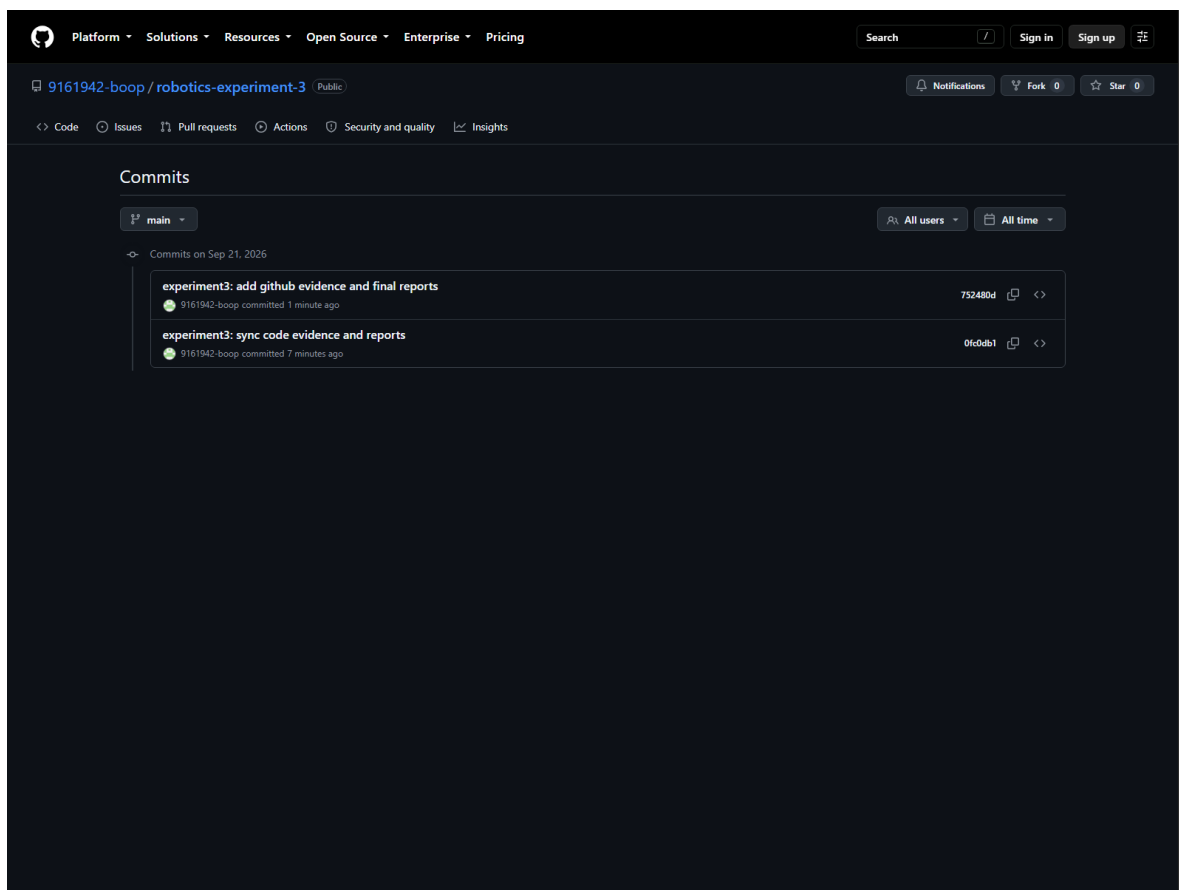


图 11 个人实验报告对应的 GitHub 实验三仓库提交历史